DEVOPS LAB – COMMANDS AND CODE

Experiments: 1, 2, 3, 6, 7 and 8

EXPERIMENT 1 – CREATE MAVEN BUILD PIPELINE IN AZURE

Git commands

```
git --version
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git init
git add .
git commit -m "First Commit"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/maven-devops-lab.git
git push -u origin main
```

azure-pipelines.yml

```
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

steps:
- task: Maven@3
  inputs:
    mavenPomFile: 'pom.xml'
    mavenOptions: '-Xmx3072m'
    javaHomeOption: 'JDKVersion'
    jdkVersionOption: '11'
    jdkArchitectureOption: 'x64'
    goals: 'package'
```

Azure DevOps GUI: Create/select project DevOps → Pipelines → New pipeline → GitHub → select repository → Maven template → Save and run.

EXPERIMENT 2 – RUN REGRESSION TESTS USING MAVEN BUILD PIPELINE IN AZURE

JUnit dependency for pom.xml

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>
```

MainTest.java

```
package org.example;

import org.junit.Test;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

import static org.junit.Assert.assertEquals;

public class MainTest {
    @Test
    public void testMainMethod() {
        ByteArrayOutputStream outContent = new ByteArrayOutputStream();
        PrintStream originalOut = System.out;
        System.setOut(new PrintStream(outContent));

        try {
            Main.main(new String[]{});
            assertEquals("Hello and welcome!", outContent.toString().trim());
        } finally {
            System.setOut(originalOut);
        }
    }
}
```

Run test

```
mvn clean test
```

Git commands

```
git init
git add .
git commit -m "Add regression tests"
git branch -M main
git push -u origin main
```

azure-pipelines.yml

```
trigger:
- main
```

```
pool:
  vmImage: 'ubuntu-latest'

steps:
- task: Maven@3
  inputs:
    mavenPomFile: 'pom.xml'
    mavenOptions: '-Xmx3072m'
    javaHomeOption: 'JDKVersion'
    jdkVersionOption: '11'
    jdkArchitectureOption: 'x64'
    goals: 'clean test'

- task: PublishTestResults@2
  inputs:
    testResultsFiles: '**/surefire-reports/TEST-*.xml'
    testRunTitle: 'JUnit Test Results'
    failTaskOnFailedTests: true
```

Azure DevOps GUI: Pipelines → New pipeline → GitHub → select RegressionTesting → Maven template → save and run on main.

EXPERIMENT 3 – INSTALL JENKINS USING ORACLE VM

Update and install Java

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
```

Add Jenkins repository and install

```
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | sudo tee
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins
```

Start and check Jenkins

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins --no-pager
```

Optional startup timeout fix

```
sudo systemctl edit jenkins
```

```
[Service]
TimeoutStartSec=300
```

```
sudo systemctl daemon-reload
sudo systemctl reset-failed jenkins
sudo systemctl start jenkins
```

Unlock Jenkins

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Browser: http://localhost:8080

GUI: Install suggested plugins → Create administrator user → Finish setup → Jenkins dashboard.

EXPERIMENT 6 – CREATE A SIMPLE ANSIBLE PLAYBOOK

Create directory and file

```
cd ~
mkdir ansible-practice1
cd ansible-practice1
nano playbook01.yml
```

playbook01.yml

```
---
- hosts: localhost
  tasks:
    - name: Print message
      debug:
        msg: Hello Ansible World
```

Run playbook

```
ansible-playbook -i localhost, -c local playbook01.yml
```

EXPERIMENT 7 – BUILD A SIMPLE APPLICATION USING GRADLE

Install and verify

```
sudo apt update
sudo apt install default-jdk
sudo apt install gradle
java -version
gradle --version
```

Create and initialize project

```
mkdir java_application
cd java_application
gradle init
ls -la
```

Run and build application

```
./gradlew run
./gradlew build
```

EXPERIMENT 8 – INSTALL ANSIBLE AND CONFIGURE ROLES

Install Ansible

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install ansible
ansible --version
```

Create directory and roles

```
mkdir -p ~/ansible_exp8
cd ~/ansible_exp8
ansible-galaxy init prerequisites
ansible-galaxy init postgresql
ansible-galaxy init apache
```

prerequisites/tasks/main.yml

```
---
- name: Install Git
  apt:
    name: git
    state: present
    update_cache: yes
```

postgresql/tasks/main.yml

```
---
- name: Install PostgreSQL
  apt:
    name: postgresql
    state: present
    update_cache: yes

- name: Start PostgreSQL service
  service:
    name: postgresql
    state: started
    enabled: yes
```

apache/tasks/main.yml

```
---
- name: Install Apache web server
  apt:
    name: apache2
    state: present
    update_cache: yes

- name: Start Apache service
  service:
    name: apache2
    state: started
    enabled: yes
```

site.yml

```
---
- name: Configure web application infrastructure
  hosts: localhost
  connection: local
  become: yes

  roles:
    - prerequisites
    - postgresql
    - apache
```

Execute and verify

```
ansible-playbook site.yml --syntax-check
sudo ansible-playbook -i localhost, -c local site.yml
git --version
systemctl is-active postgresql
systemctl is-active apache2
```